© 2019. This manuscript version is made available under the CC-BY-NC-ND 4.0 license https://creativecommons.org/licenses/by-nc-nd/4.0/. The final version was published in Information Processing & Management 56(5): 1753-1770, https://doi.org/10.1016/j.ipm.2019.05.011

Querying XML Documents using Prolog Engines: when is this a good idea?

Fábio Gomes dos Santos[1], Leonardo Machado, Rafael Pinheiro, Aline Paes, Vanessa Braganholo*

Instituto de Computação, Universidade Federal Fluminense
Niterói, RJ, Brazil

Abstract

XML has become a universal standard for information exchange over the Web due to features such as simple syntax and extensibility. Processing queries over these documents has been the focus of several research groups. In fact, there is broad literature in efficient XML query processing which explore indexes, fragmentation techniques, etc. However, for answering complex queries, existing approaches mainly analyze information that is explicitly defined in the XML document. A few work investigate the use of Prolog to increase the query possibilities, allowing inference over the data content. This can cause a significant increase in the query possibilities and expressive power, allowing access to non-obvious information. However, this requires translating the XML documents into Prolog facts. But for regular queries (which do not require inference), is this a good alternative? What kind of queries could benefit from the Prolog translation? Can we always use Prolog engines to execute XML queries in an efficient way? There are many questions involved in adopting an alternative approach to run XML queries. In this work, we investigate this matter by translating XML queries into Prolog queries and comparing the query processing times using Prolog and native XML engines. Our work contributes by providing a set

*Corresponding author
Email addresses: fabiogomes@id.uff.br (Fábio Gomes dos Santos), lmachado@ic.uff.br (Leonardo Machado), rpinheiro@id.uff.br (Rafael Pinheiro), alinepaes@ic.uff.br (Aline Paes), vanessa@ic.uff.br (Vanessa Braganholo)
[1]Fábio Gomes dos Santos is currently at Globo.com

of heuristics that helps users to decide when to use Prolog engines to process a given XML query. In summary, our results show that queries that search elements by a key value or by its position (simple search) are more efficient when run in Prolog than in native XML engines. Also, queries over large datasets, or that searches for substrings perform better when run by native XML engines.
Keywords: XML, XQuery, Prolog, Inference, Query Processing

1. Introduction

XML has been widely used for information exchange over the Web due to features such as simple syntax and extensibility. Processing queries over these documents has been the focus of several research groups. In fact, there is broad literature in efficient XML query processing which explores query optimization [17, 56, 28, 14, 27], indexes [37, 4, 54], fragmentation techniques [18], etc.

The standard XML query languages, XPath [21, 48] and XQuery [15, 49] are able to query both the structure of the document and its content. Other approaches use different techniques to extract the content of XML documents. They use resources like natural language [55] or information retrieval [31, 26, 63, 25], for instance. In all of these cases, however, the query is processed over explicit data, ignoring any implicit information that may be obtained from it. For several applications, however, this is a limitation. Consider for instance an XML document containing information about kinship, as shown in Figure 1. For each person, the document stores the person name, birth date, mother name and father name. Based on this information, some applications need to infer data about other relationships such as grandfather, uncle, sibling, etc. This is not possible in XPath, and not easily done in XQuery, since it requires a recursive query that is usually not efficient.

To make this easier, alternative approaches have been proposed [16, 53, 12, 5, 38]. They translate the XML document into Prolog facts and rules, aiming at providing ways to get more information from XML datasets by allowing inference. However, they all require the XML document to be translated into

```
<family>
   <person birth_date="1978-12-03">
      <name>Mary</name>
      <mother>Martha</mother>
      <father>John</father>
   </person>
   <person birth_date="1951-06-25">
      <name>Martha</name>
      <mother>Joan</mother>
      <father>Joseph</father>
   </person>
   <person birth_date="1930-04-20">
      <name>Joan</name>
      <mother>Ann</mother>
      <father>Paul</father>
   </person>
</family>
```

Figure 1: Example of an XML document, based on the kinship domain

Prolog or other logic-based language. On the one hand, this solves the inference problem, but it creates another: what about regular queries (those that do not require inference)? What would be the impact on query processing in terms of performance? What kind of queries could benefit from the Prolog translation? There are many questions involved in adopting an alternative approach to run XQuery queries. In this work, we investigate this matter by translating XML queries into Prolog queries and comparing the query processing runtimes using Prolog and native XML engines.

To guide our investigation, our main goal is to answer two research questions:

RQ1 Are Prolog engines comparable to XQuery engines in terms of performance?

RQ2 When is it a good choice to use Prolog engines to process XQuery queries?

To answer these questions, the first challenge resides in translating XML into Prolog facts. Among the alternatives available in the literature [16, 53, 12, 5, 38], in this work, we adopt and improve our previous work [38], which is called XML Inference. The reasons for this choice are the following: (i) it is one of the few approaches in the literature [38, 5] that does not use a single

Prolog fact to represent the entire XML document. In fact, XML Inference uses different facts to represent different elements, which allows us to write simpler rules to query the document; (ii) when compared to [5], XML Inference uses plain values instead of lists, which also contributes to simpler queries; (iii) the implementation is publicly available at `https://github.com/dew-uff/xml-inference`.

The main challenge, however, resides on how to translate the XQuery queries into equivalent Prolog queries. In fact, we did not find any research work or prototype that provides a translation algorithm. However, before investing time in a translation algorithm, which would be very complex due to the diversity of the XQuery constructs [15], we must analyze the query processing performance to decide if the effort would pay off. Thus, in this work, XQuery queries are translated to Prolog in an *ad-hoc* manner.

The queries and XML documents are provided by the XBench benchmark [62]. XBench provides different types of datasets. The dataset can be composed of a single document (SD) or multiple documents (MD) and can be data-centric (DC) or text-centric (TC). Over the DC/SD, TC/SD and TC/MD datasets, we execute the set of queries provided by XBench using the following query processing engines: Galax[2] (a command line XQuery processor), Sedna [32], eXist [42] and BaseX [35] (three native XML DBMS), and two different Prolog engines, SWI [61] and Yap [24].

The remaining of this article is organized as follows. Section 2 discusses related work. Section 3 presents and improves XML Inference [38], the XML-to-Prolog translation method we use in this work. Section 4 presents the experimental setup we used to answer the two research questions. Section 5 discusses the experimental results and presents a set of heuristics that can be used to decide when to use Prolog engines to process XML queries. Finally, Section 6 concludes the article.

[2] `http://galax.sourceforge.net/`

```
<a>                                  <a x="1">
  <b>text1</b>                         <b>text1</b>
  <c>text2</c>                         <c>text2</c>
</a>                                 </a>
```

Figure 2: Example of a simple XML document `sample.xml` (left) and a variation with an attribute (right)

2. Related Work

There is a broad literature about XML query processing. Several papers discuss how to implement various types of queries: structural queries [58, 56, 14], keyword queries [67, 3, 66], etc. Other approaches use different techniques to extract content of XML documents. They use resources like natural language [55] or information retrieval [31, 26, 63, 25], for instance. Additionally, several techniques have been used to accelerate XML query processing, such as indexing [4] and parallel query processing [10, 18, 27].

There are also several papers and reports that investigate the performance of XQuery processors [57, 2, 41, 13]. They use existing benchmarks to compare query processing times of different XML engines. Manegold [41] use several benchmarks to evaluate both the quality of the benchmarks and the performance of several XML engines. Afanasiev, Franceschet, and Marx [2] propose a platform to benchmark XML engines that supports several existing benchmarks. They also present experimental results comparing several XML engines using the proposed platform. Finally, Bamford et al. [13] analyze how the XML engines support XQuery constructs, including updates. However, none of them investigates the use of Prolog to process queries.

Researchers have been investigating the integration of Logic Programming and the Database field for quite some time. In 1977, Gallaire and Minker organized a Workshop on Logic and Databases [43] that got together several researchers that were interested in these subjects. A concrete result of the efforts in bringing Logic Programming and Databases together is Datalog, a declarative logic programming language that has been used to query relational databases [1]. Prolog has also been used for the same purpose [45]. Using logic

languages to query and represent (semi-)structured data such as XML is then a natural unfold [16, 53, 22, 12, 23, 5, 61, 38, 11, 9, 40]. In fact, XPath and XQuery have already been shown to fit nicely in a logic programming language (TOY) [8, 7].

In this context, there are two different lines of work. The first one consists of a set of logic based languages that aim to process XML [19, 23]. Xcerpt is an XML processing language that, like its predecessor UnQL [20], uses patterns instead of paths to access XML elements. XCentric [23] is a language that, as the authors claim, is intended to be easy to use by logic programmers. Using XCentric, XML processing programs can be written in a more declarative way when compared to standard Prolog libraries that process XML.

On the second line of work, there are several approaches that translate XML into Prolog, which are discussed next. Boley [16] investigates the use of XML in formal specifications and programs and establishes a strong relationship between logic programming and XML. He investigates both XML to Prolog translation, and Prolog to XML translation. XML to Prolog translation, which is of special interest to this paper, uses nested facts to represent the XML hierarchy. As an example, the XML document of Figure 2(left) would be represented in Prolog as `a(b(text1),c(text2))`. Representing attributes would require nesting terms, which is not supported by Prolog. Boley also investigates how to query the resulting XML document by using XQL [47].

The work in [53] proposes *Field-Notation*, a model to represent XML documents in Prolog. Fied-Notation uses association lists to represent data in Prolog. This means that the entire XML document is placed in a list. Elements inside this list are nested and represented as attribute/value pairs. As an example, the document of Figure 2(right) would be represented as follows: `a:[x:1]:[[b:text1]:[],[c:text2]:[]]`. The empty lists [] denote that b and c have no child elements. Seipel [53] also presents a translation for path queries, and an algorithm to execute these queries over the translated data. However, there is no performance evaluation.

Coelho and Florido [22] infer types in XML documents by using logic pro-

gramming. Their work uses a DTD to guide the translation process. The document is translated into a single fact, and attributes are represented using lists. As an example, the document of Figure 2(right) would be translated as `a([attribute('x','1')],b([],'text1'),c([],'text2'))`.

The SWI-Prolog SGML/XML parser [60], which is part of the Prolog processor SWI-Prolog [61] converts SGML, XML and HTML files into Prolog facts. Similar to the aforementioned work, the translation also uses a single list to represent the document. As an example, the document of Figure 2(right) would be translated as `[element(a, [x='1'], [element(b, [], [text1]), element(c, [], [text2])])]`. As a common point, all of these translations [16, 53, 22, 61] generate large Prolog facts, which complicates query composition.

Almendros-Jiménez, Becerra-Terón and Enciso-Baños [5] translate XML documents to Prolog facts and rules. Leaf nodes are represented as facts, and internal nodes are represented as rules. As an example, the document of Figure 2(right) would be translated as follows:

```
a(atype(A, [X], NodeA, 1) :-
   b(NodeB, [NodeB|NodeA], 2),
   c(NodeC, [NodeC|NodeA], 2),
   x(X, NodeX, 3).
x('1', 1, 3).
b('text1', [1,1], 2).
c('text2', [2,1], 2).
```

This is done with the goal of implementing the XPath language [21] using Logic Programming. Initial performance evaluation shows execution times of three XPath queries, but no comparison with native XML query processors is given. Another work of the same research group [6] investigates how to integrate the XQuery language and logic programming. They build a complete system to execute XQuery queries on Prolog engines that includes the translation of the XML document into Prolog facts (the same as above), and also the translation of the queries into Prolog rules. However, this work does not present a heuristic

to help users decide the better approach to be used to process queries and there is no mention of a performance evaluation.

In previous work, we translate XML documents into Prolog facts using different facts for different XML elements [38]. XML content is translated into Prolog facts and the schema is translated into Prolog rules to make querying the document content easier. We then use this translation as a basis for an algorithm to translate XPath queries into Prolog [51]. However, no experimental evaluation was conducted at that time.

A limitation of our previous translation method [38] is that it does not guarantee that the order of the leaf nodes in the original document is preserved. As an example, in the XML document of Figure 1, elements *person* have children *name*, *mother*, and *father*. When we query for *person*, elements *name*, *mother*, and *father* should appear in the result inside *person*, respecting the document order (*name* followed by *mother*, followed by *father*). However, this is not possible if the Prolog facts do not preserve the order information of the leaf nodes. Additionally, our previous translation method was not able to differentiate elements from attributes, so it was not possible to correctly reconstruct the original XML document in a query answer. To make these possible, in the next section we present an extended version of our previous translation rules [38] that guarantees the preservation of the leaf nodes' order and the distinction of elements and attributes.

3. XMLInference: Translating XML-to-Prolog

In this section, we present our approach to translating XML documents into Prolog facts. This approach is an extension of our previous work [38]. Our approach, XMLInference, processes data contained in the XML structure with the goal of inferring new information. It enables the formulation of more complex queries to obtain either explicit information or information deducted through inference rules (implicit information).

The translation rules generate several facts for a single XML document. Each

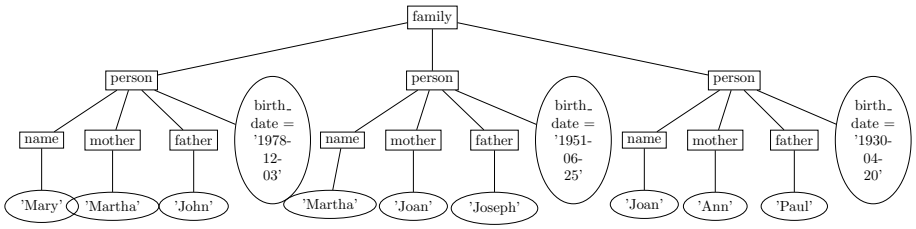


Figure 3: Tree representation of the document of Figure 1

XML element is transformed into a predicate, and its content is transformed into constants of these predicates. To relate distinct predicates, we use a Prolog constant as an extra identifier. This allows connecting facts that represent parent/child relations in the XML document.

During the translating process, the document is traversed from top to bottom. Each element found in this path is translated by a translation rule. The translation rules are based on the type of the element that is currently being translated: (i) document root; (ii) complex element; (iii) atomic element; or (iv) mixed element. We also have a translation rule for attributes. We discuss the translation rules for each of these types as follows.

To illustrate the translation process, we use the XML document of Figure 1. Its tree representation is shown in Figure 3. As mentioned in the introduction,the document contains kinship information. The Prolog facts corresponding to this document are presented in Figure 4.

Document root. The root is the only element in the XML document that does not have a parent. In most cases, the root is a complex element (unless it is a rare case of a document that consists of a single simple element). A complex root is translated as a fact with the element name and a single argument that is equal to an automatically generated *id*. The main purpose of this *id* is to serve as a connection with the child elements of the root. In our example, the root element *family* is translated as `family(1)`, where `1` is its *id* (Figure 4, line 1).

Complex elements. Complex elements are those that have children, but no text. For elements of this type, an identifier is automatically generated to relate

```
 1. family(1).
 2. person(1, 2).
 3. person_attribute_birth_date(2, 3, '1978-12-03').
 4. name(2, 4, 'Mary').
 5. mother(2, 5, 'Martha').
 6. father(2, 6, 'John').
 7. person(1, 7).
 8. person_attribute_birth_date(7, 8, '1951-06-25').
 9. name(7, 9, 'Martha').
10. mother(7, 10, 'Joan').
11. father(7, 11, 'Joseph').
12. person(1, 12).
13. person_attribute_birth_date(12, 13, '1978-12-03').
14. name(12, 14, 'Joan').
15. mother(12, 15, 'Ann').
16. father(12, 16, 'Paul').
```

Figure 4: Prolog translation of the XML document of Figure 1

the children with their parent. As the result of the translation, XMLInference generates a fact with the complex element name and two arguments: the parent's *id* and the element's *id*. In our example, element *person* is translated as `person(1, 2)` (Figure 4, line 2), where `1` is the parent *id* (*family*) and `2` is the element's *id*. Note that *person* at line 7 is translated as `person(1, 7)`, where `1` is the parent *id* (all *person* elements in this document have the same parent) and `7` is the element's *id*.

Atomic elements. Atomic elements are those that have textual content as their single child in the XML tree. Thus, an atomic element is translated as a Prolog fact named after the element's name. The arguments, in this case, are the parent *id*, the element *id* and the element content itself. As an example, element *name* is translated as `name(2, 4, 'Mary')` (Figure 4, line 4), where `2` is the parent *id* (*person*) and `4` is the element's *id*. The *id*s are sequentially generated. The document root has *id* 1, and its first child has *id* 2.

Note that, since atomic elements do not have children, there would be no need to add an element *id* in the translation to serve as a parent of non-existing children. In fact, in the first version of our translation rules, there was no such

id [38]. In this paper, we extend this translation rule by adding the element *id* to preserve the order among the siblings. Since the *id* is automatically generated and elements are processed in the order they appear in the document, we guarantee that the *id* can be used to preserve the order. In our example, it is easy to see that *mother* is a sibling of *name* (as they have the same parent *id* `2`), and that *name* appears before *mother* in the XML document (as the element *id* of *mother* (`5`) is greater than that of *name* (`4`) – see lines 4 and 5 of Figure 4).

Attributes. The translation of attributes follows the same logic we use for atomic elements. The only difference is the fact name, which is the element name that contains that attribute, followed by `_attribute_` followed by the attribute name. As an example, the attribute *birth_date* of element *person* of Figure 1 is translated as a Prolog fact named `person_attribute_birth_date`. Similar to atomic elements, this fact has also the parent *id*, attribute *id* and its value. The translation of attribute *birth_date* is `person_attribute_birth_date (2, 3, '1978-12-03')`, where `2` is the parent *id*, `3` is the attribute *id* and `'1978-12-03'` is the attribute value (line 3 of Figure 1).

Mixed elements. Mixed elements are those that combine textual content and children elements. Their translation rule is similar to that of complex elements, except that the textual context of these elements generate facts named `xmlmixedElement` (since they do not have names). The fact has the parent *id*, the element *id*, and its textual content as terms. As an example, consider the XML document in Figure 5(top) and its corresponding Prolog translation in Figure 5(bottom). In this figure, *history* is a mixed element, which is translated as a complex element (Figure 5, line 2). The content of this element is text (three fragments) and two children (*city* and *country*). The translation of text fragments produces three `xmlmixedElement` facts, one for each text that appears as content of element *history* (Figure 5 lines 3, 5 and 7). The children of the mixed element are translated according to their type. In the case of the example, they are all atomic elements (Figure 5 lines 4 and 6).

```
<family>
  <history>The ancestors of the family came from <city>Venice</city>,
  in <country>Italy</country> in 1903.
  </history>
</family>
```

```
1. family(1).
2. history(1, 2).
3. xmlmixedElement(2, 3, 'The ancestors of the family came from').
4. city(2, 4, 'Venice').
5. xmlmixedElement(2, 5, ', in').
6. country(2, 6, 'Italy').
7. xmlmixedElement(2, 7, 'in 1903.').
```

Figure 5: Example of XML document with mixed element (top) and its Prolog translation (bottom)

The details of the translation algorithm are shown in Figure 6. We use a stack (*parents*) to control the parent *id*s, and another (*mixed*) to know if a text content belongs to a mixed element. The input XML document is processed using a SAX Parser, and the functions *start_document(), end_document(), start_element(), end_element(),* and *characters()* are called by the SAX Parser do the translation. The result is written into a text file (we refer to this file as *facts* file in the algorithm).

In the *start_element* function, facts are written to the *facts* file as soon as the element type is detected. However, an exception is made for atomic elements, since a fact that represents atomic elements need information that is in the start tag (the element name, which is provided by the parser in the *start_element* function) and also information that is provided by the parser in the *characters* function. This means that this fact starts to be written in the *start_element* function, and ends at the *characters* function. Note that facts referring to attributes are also written in the *start_element* function. If the atomic element has attributes, our algorithm makes sure the attribute fact is written before the element fact, to guarantee the element fact will not be truncated at the resulting *facts* file.

```
translate(xml):
   parse xml using SAX Parser

start_document():
   id = 1
   create empty stack parents
   create empty stack mixed
   open facts file for writing

end_document():
   close facts file

start_element(e, attributes):
   if type(e) == ROOT:
      write into facts: name(e) + "(" + id + ")./n"
      push "False" into mixed stack
   else if type(e) == COMPLEX:
      write into facts: name(e) + "(" + peek(parents) + ", " id + ")./n"
      push "False" into mixed stack
   else if type(e) == ATOMIC:
      //postpone writing this fact, so attributes can be written first
      //the rest of this fact will be written at the characters function
      temp = name(e) + "(" + peek(parents) + ", " id + ", "
      push "False" into mixed_stack
   else if type(e) = MIXED:
      write into facts: name(e) + "(" + peek(parents) + ", " id + ")./n"
      push "True" into mixed stack
   parent_id = id
   push parent_id into parents stack
   id++

   for each a in attributes:
      write into facts: name(e) + "_attribute_" + name(a)("
      write into facts: peek(parents) + ", " id + value(a) + ")./n"
      id++
   if type(e) == ATOMIC:
       write temp into facts

end_element():
   pop element of parents stack
   pop element of mixed stack

characters(text):
   text_content = text
   if not(peek(mixed)):
      write into facts: "'" + text + "')./n"
   else:
      //this text belongs to a mixed element
      write into facts: "xmlmixedElement(" + peek(parents) + ", "
      write into facts: id + ", '" + text + "')./n"
```

Figure 6: Translation algorithm

4. Experimental Setup

To answer our research questions, we conducted an experiment using the XBench benchmark [62]. XBench provides four different types of benchmark datasets, together with a set of corresponding XQuery queries. In our evaluation, we used three of these datasets: data-centric single document (DC/SD), text-centric single document (TC/SD), and text-centric multiple document (TC/MD). Each of these datasets was generated using two different scale factors to obtain datasets of sizes 10MB and 1GB. In a single document database, the whole dataset is stored in a single file, while the multiple document database has more than one file composing the dataset. When translated to Prolog, an MD dataset is transformed into an SD dataset, since all the facts that represent the documents that compose the dataset are stored in a single Prolog file. Both multiple document and single document can be data-centric or text-centric. In the former, the majority of the information is data, while in the later it is text.

The three selected datasets together provide 52 queries. These queries are distributed between datasets as follows: 16 queries for the data-centric single document (DC/SD), 17 queries for the text-centric single document (TC/SD), and 19 queries for the text-centric multiple document (TC/MD). We discarded one query from the TC/SD dataset (Q10) because its textual description does not match its XQuery implementation. We then ended up with 51 queries from the Benchmark.

Since the motivations that lead us to find out the benefits of using Prolog engines to execute XML queries was the aim of using inference over XML data, we also tested a recursive query and evaluated its performance. Figure 7 shows a recursive XQuery that retrieves a network of co-authors of an author named "John Doe," recursively. As an example of what this query does, assume the XML document states that author A co-authored a book with author B, and author B co-authored a book with authors C and D, and author C co-authored a book with author E. If we ask for the co-author network of author A, the query will return B, C, D, E. If we ask for the co-author network of author C, the query

will retrieve authors D and E. This query was run over a modified version of the DC/SD 10MB dataset. The modification was needed to make sure the recursive query would retrieve a non-empty result. We then made two variations of the DC/SD 10MB dataset: one that modifies the document by replacing the third author of an item (if it exists) to "John Doe," but only for items that satisfy the condition `position(.) mod 10 = 0`. We call this variation *DC/SD 10MB item 10*. The second variation replaces the third author on the original DC/SD 10MB dataset, but only for items that satisfy the condition `position(.) mod 5 = 0`. We call this variation *DC/SD 10MB item 5*.

We ran these queries using six different query engines: two Prolog engines and four XQuery engines. As Prolog engines, we chose SWI [61] (due to its popularity) and YAP [24], due to its well known good performance. As XQuery engines, we chose three native XML databases (eXist [42], Sedna [32] and BaseX [35]), and one engine that works over XML files stored on the file system (Galax [29]). For the XQuery engines, the datasets were stored in the file system (for Galax), and in Sedna, eXist and BaseX. Note that native XML databases fully index the documents when they are loaded into the database, including the textual content of elements, and we kept this default indexing behavior in our experiments.

For the Prolog engines, we translated the datasets to Prolog facts using the translation rules of Section 3. Our implementation of XMLInference is in Java and is publicly available at `https://github.com/dew-uff/xml-inference`. We use a SAX Parser to parse the XML document and generate the corresponding Prolog facts. Also, to avoid memory problems when dealing with large XML documents, we use a C library that uses the SWIProlog API [59]. Thus, SWI is a requirement for XML Inference to work.

The queries were manually translated to Prolog in an ad-hoc manner. The goal is to evaluate their performance, and, if the results are positive, then invest in an automatic translation algorithm in future work. Figure 8 shows an example of a query and its corresponding translation to Prolog. This is the query $Q2$ of the Data-Centric Single Document dataset. It returns the title of the item

```
declare function local:network($stack, $seen) {
  if(empty($stack)) then $seen
  else (
    let $current_author := $stack[1]
    let $coauthors :=
        for $a in $current_author/../author[not(name = $seen/name)]
        return  $current_author/../../..//authors/author[name = $a/name]
    return local:network(($coauthors, $stack[position() > 1]),
           ($seen, $coauthors))
  )
};

let $a := /catalog/item/authors/author
         [name/first_name = "John" and name/last_name = "Doe"]
return
   distinct-values(local:network($a[1], ())/name)
```

```
search_path_author(FName) :-
   once(first_name(Index_name, _, FName)),
   middle_name(Index_name, _, MName),
   last_name(Index_name, _, LName),
   name1(Index, Index_name),
   author(Index_author, Index),
   search_path_author([(Index_author, FName, MName, LName)], []).

search_path_author([], _).
search_path_author([(Index_author, FName, MName, LName)|Frontier], Visited) :-
   write(FName), write(MName), write(LName), nl,
   findall(Ind, (author(Ind, Ind1), name1(Ind1, Ind2),
      first_name(Ind2, _, FName),
      middle_name(Ind2, _, MName), last_name(Ind2, _, LName)), Indexes),
   get_children(Indexes, [], Children),
   append([(Index_author, FName, MName, LName)], Visited, Visited1),
   add_frontier(Children, Frontier, Visited1, Frontier1),
   search_path_author(Frontier1, Visited1).

get_children([], Children, Children).
get_children([I1|Is], Children, All_children) :-
   findall((I1, FN, MN, LN),
      (author(I1, OIndex), name1(OIndex, OIndex_name),
         first_name(OIndex_name, _, FN), middle_name(OIndex_name, _, MN),
         last_name(OIndex_name, _, LN)), Authors),
      append(Children, Authors, Children1),
      get_children(Is, Children1, All_children).

add_frontier([], Frontier, _, Frontier).
add_frontier([(_, FN, MN, LN)|OAuthors], Frontier, Visited, Frontier1):-
   memberchk((_, FN, MN, LN), Frontier),!,
   add_frontier(OAuthors, Frontier, Visited, Frontier1).
add_frontier([(_, FN, MN, LN)|OAuthors], Frontier, Visited, Frontier1):-
   memberchk((_, FN, MN, LN), Visited),!,
   add_frontier(OAuthors, Frontier, Visited, Frontier1).
add_frontier([H|OAuthors], Frontier, Visited, Frontier1):-
   append(Frontier, [H], Frontier_temp),
   add_frontier(OAuthors, Frontier_temp, Visited, Frontier1).
```

Figure 7: Recursive query in XQuery (top) and its Prolog translation (bottom)

```
for $item in doc("doc.xml")/catalog/:item
where $item/authors/author/name/first_name = "Ben"
return
   $item/title
```

```
pQ02 :- findall(X,first_name(X,_,'Ben'),LIST), pQ02_1(LIST).
pQ02_1([]).
pQ02_1([H|T]) :- name(Y,H), author(Z,Y), authors(A,Z), title(A,_,TITLE),
             print('<title>'), print(TITLE), print('</title>'), nl, pQ02_1(T).
```

Figure 8: Example of XQuery and its corresponding ad-hoc translation to Prolog

authored by "Ben." The Prolog translation first builds a list with all first names "Ben." Then, it takes advantage of the knowledge of the document schema: it builds the path traversal */authors/author/name/first_name* by linking the Prolog facts by the parent id. It does the same to relate *item* (parent of authors) with *title*, which is the output of the query. Note that the output of the query is built using the `print` Prolog command. In the case of the example, we print the `<title>` start tag, and the title content is printed using `print(TITLE)`. Finally, we print the close tag `</title>` and continue to process the remaining of the list. Queries with more complex output require several print commands. The translation of the remaining 50 queries is available at [50, p. 71-131].

While XML query processors use traditional query optimization techniques [56, 14] like translating the queries into an algebraic representation [30, 33, 36, 46, 52, 64, 65] and manipulating these representations, using indexes [4, 54], etc, our approach does not benefit from these techniques. On the contrary, we translate XQuery into Prolog, and the query processing is fully handled by the Prolog engine. Modern Prolog engines like the ones we use in our experiments index terms to speed up queries [44]. This is achieved by compiling the knowledge base of facts and rules. We compiled the knowledge bases we used in our experiments, both on SWI and on YAP.

The experiments with the 10MB datasets were performed on an Intel Dual Core 2.16 GHz machine, with 2 GB of RAM memory and 32-bit Linux (Ubuntu) operating system. Each query was run 11 times on each tool and the first run

was discarded to simulate a real environment where some data resides in cache. Experiments with the 10MB datasets were run over SWI, YAP, Sedna, eXist and Galax. The experiments with the 1GB datasets were run on a 6 core Intel i7-8700K 3.7 GHz machine, with 16 GB of RAM memory and 10 TB HD running 64-bit Ubuntu. Each query was run 6 times, and the first run was also discarded. We took the average of the remaining times. Experiments with the 1GB datasets were run over SWI, YAP, Sedna and BaseX. Since the results for 10MB were not good for Galax (it does not index the documents) and eXist (Sedna outperformed eXist in several queries), we dropped Galax and eXist in this larger dataset, and added BaseX, which is well known for its good performance.

5. Results and Discussion

In the following subsections, we discuss the results and answer the two research questions that guide this work. Section 5.1 shows the experimental results for the 10MB datasets, while Section 5.2 shows the results for the 1GB datasets. Section 5.3 shows the results for the recursive query. Then, Sections 5.4 and 5.5 presents the answer to the research questions.

5.1. Results for the 10MB datasets

Figure 9 shows the execution times for the 16 queries of the TC/SD 10MB dataset in SWI, YAP, Sedna and eXist. Galax performance was much worse than the performance of the other query engines, and so we do not show it in our graphs. Results for Galax can be found at [50, p. 71-131]. Note that Prolog engines performed better than XML engines in 5 queries (Q2, Q3, Q4, Q13, and Q14). This means that both SWI and YAP performed better than Sedna and eXist. In Q8 and Q9, there was a tie between SWI and Sedna. For Q11 and Q19, SWI performed better than both the XML engines, but YAP did not.

For the DC/SD 10MB dataset, results were much better (Figure 10). In this dataset, Prolog engines outperformed the XML engines in 7 out of 16 queries

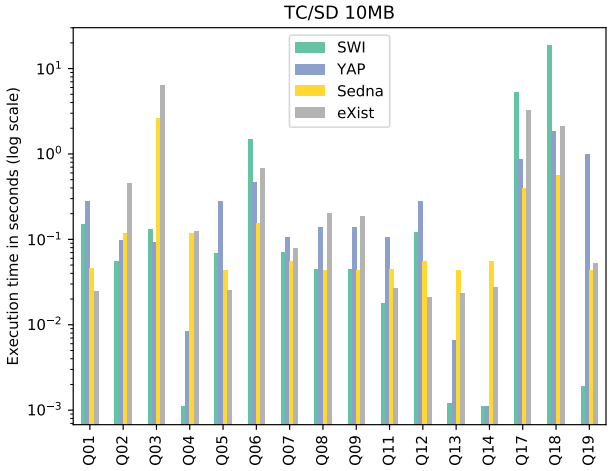


Figure 9: Execution times on the TC/SD 10MB dataset

(Q2, Q4, Q9, Q10, Q11, Q14, and Q19). Additionally, one of the Prolog engines performed better than both the XML engines in 9 queries (Q1, Q3, Q5, Q6, Q7, Q8, Q12, Q17, and Q20). SWI performed better in 8 of these queries. The exception was Q20, where YAP performed better.

Figure 11 shows the results for the TC/MD 10MB dataset. Out of 19 queries, Prolog engines performed better than both XML engines in 12 queries (Q1, Q2, Q3, Q4, Q5, Q8, Q9, Q10, Q11, Q14, Q15, and Q19). In three other queries (Q6, Q7, and Q13), one of the Prolog engines performed better than both XML engines. YAP performed better in Q6 and Q7, while SWI performed better in Q13.

5.2. Results for the 1GB datasets

Figure 12 shows the performance for the TC/SD 1GB dataset. For this dataset, Sedna was not able to process Q3. It returns an error and aborts the execution. Overall, XML engines performance were superior when compared to the Prolog engines, except for queries Q4 and Q14. Note that in the 10MB TC/SD dataset, these queries also performed better in Prolog engines than in XML engines. For the other queries that outperformed XML engines in the

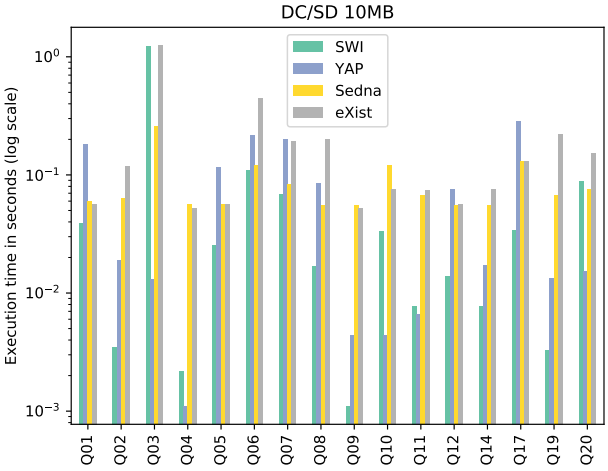


Figure 10: Execution times on the DC/SD 10MB dataset.

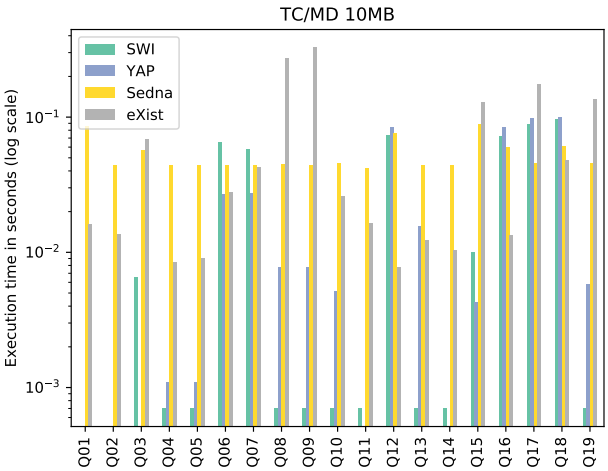


Figure 11: Execution times on the TC/MD 10MB dataset

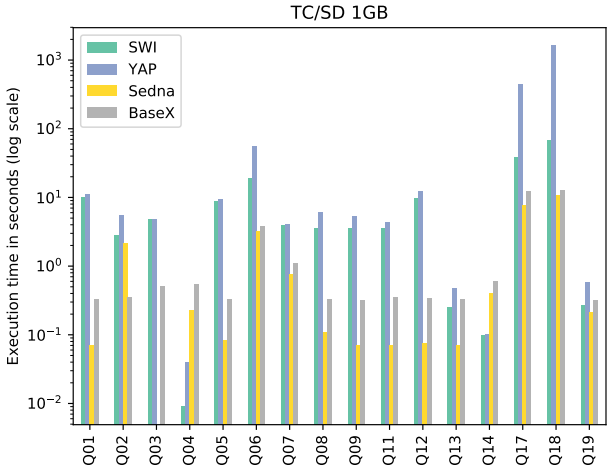


Figure 12: Execution times on the TC/SD 1GB dataset. Sedna returns an error and aborts the execution of Q3.

10MB dataset, the dataset size impacts the performance negatively.

Results for the DC/SD 1GB dataset are shown in Figure 13. For this dataset, XML engines performed better than Prolog engines in all of the queries, except for Q3, Q4, and Q17. In Q11, YAP outperforms both Sedna and BaseX, but SWI does not. When we look only at the XML engines, Sedna outperforms BaseX in 12 of the 16 queries. When compared to the DC/SD 10MB dataset, the queries for which Prolog engines perform better than XML engines are a subset of the queries where the same occurred in the 1GB dataset. Thus, the size of the dataset impacts in the performance of the Prolog engines for queries Q1, Q2, Q5, Q6, Q7, Q8, Q9, Q10, Q12, Q14, Q19, and Q20.

Figure 14 shows the performance for the TC/MD 1GB dataset. In 10 out of 19 queries (Q1, Q2, Q3, Q4, Q5, Q10, Q11, Q14, Q15, and Q19), Prolog engines performed better than XML engines. YAP performs better than SWI in most of these queries, except for Q5 and Q19. When compared to the 10MB dataset, the results are similar, with two basic differences: (i) queries Q8 and Q9 perform better in Prolog engines in the 10MB dataset, while in the 1GB dataset, they perform better with XML engines; (ii) In the 10MB dataset, SWi

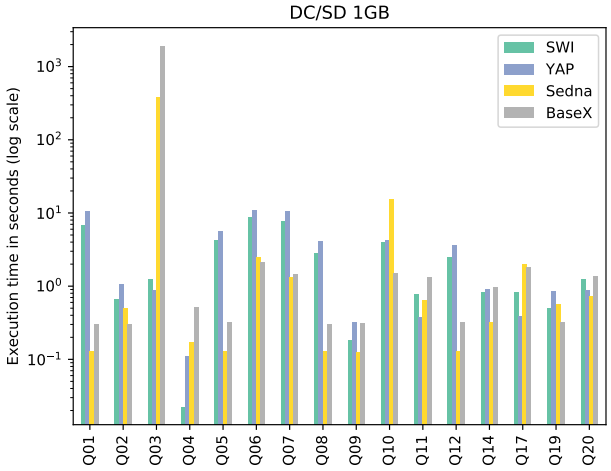


Figure 13: Execution times on the DC/SD 1GB dataset

performs better than YAP for most of the queries, while in the 1GB dataset, YAP performs better. This means that YAP handles large datasets better than SWI. Nevertheless, the performance of the queries of the TC/MD 1GB dataset is good with Prolog engines, unlike the results we obtained for the SD datasets. The explanation, in this case, resides in the way we handle MD datasets in Prolog. In XML, the MD dataset is composed of several small XML documents. In our approach, we represent all of these documents into a single Prolog knowledge base. The time to parse and access several XML documents when contrasted to accessing a single knowledge base proved crucial in the performance of the queries.

5.3. Results for the Recursive Query

We ran the recursive query on SWI, YAP, Sedna and BaseX using a 6 core Intel i7-8700K 3.7 GHz machine, with 16 GB of RAM memory and 10 TB HD running 64-bit Ubuntu. Figure 15 shows the results. Note that the processing time of SWI and YAP are two orders of magnitude smaller than that of BaseX. Sedna was not able to execute the query, with the message "*ERROR SE1001 - Physical plan executor stack overflow.*". We tried to increase the size of the

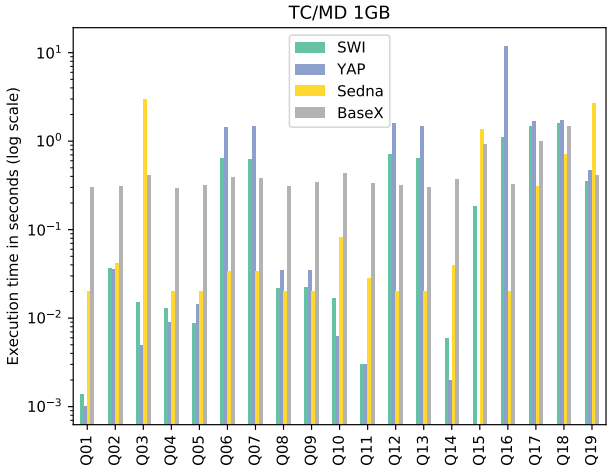


Figure 14: Execution times on the TC/MD 1GB dataset

stack in Sedna's configuration file (we used several values from 4,000 – *default value* – to 40,000), but the problem persisted.

5.4. RQ1: Are Prolog engines comparable to XQuery engines in terms of performance?

To answer this research question, we first counted the number of wins of each query engine, for each dataset size. For each query of a given dataset size, we look at the engine that took less processing time, and we count this as one win. The histogram of the wins for the 10MB datasets is shown in Figure 16. By analyzing the performance graphs on Section 5.1 and the histogram of Figure 16, it is clear that SWI performed better among the Prolog engines (27 wins), and Sedna performed better among the XML engines (7 wins). We also see that SWI wins 3.85 times more than Sedna. For the 1GB dataset, the histogram is shown in Figure 17. Here, the situation reverts. Sedna has the most wins among all of the engines (29 wins), while YAP has the most wins for the Prolog engines (10 wins).

Then, we checked whether our results are statistically significant. For this, we took the execution times for each query and proceeded a pairwise comparison

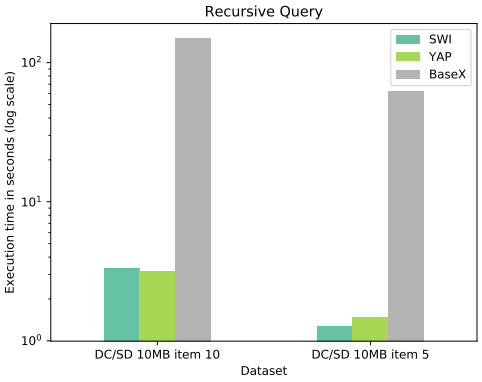


Figure 15: Execution times of the recursive query

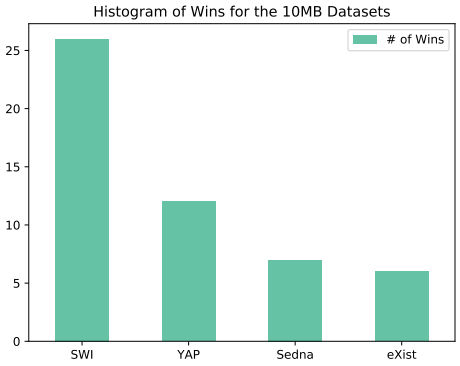


Figure 16: Histogram of wins for the 10MB datasets

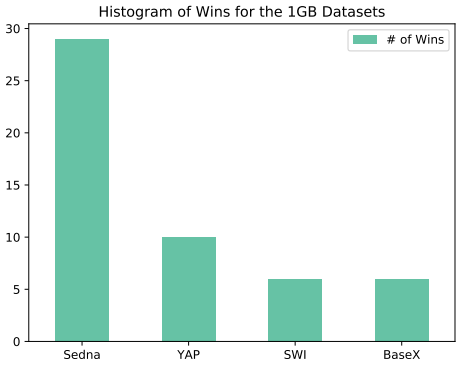


Figure 17: Histogram of wins for the 1GB datasets

Table 1: Results of the Wilcox paired test (*p-value*). Statistically significant differences are highlighted in italics

	p-value	**Conclusion**
SWI x Sedna, TC/SD 10MB	0.8948000	(inconclusive)
SWI x Sedna, DC/SD 10MB	0.0034570	SWI performs better
SWI x Sedna, TC/MD 10MB	0.0174200	SWI performs better
SWI x Sedna, all three datasets of 10MB	0.0008329	SWI performs better
YAP x Sedna, TC/SD 1GB	0.0029370	Sedna performs better
YAP x Sedna, DC/SD 1GB	0.0183900	Sedna performs better
YAP x Sedna, TC/MD 1GB	0.0486400	YAP performs better, but effect size is negligible
YAP x Sedna, all three datasets of 1GB	0.0112700	Sedna performs better

of the results of the two approaches with more wins on the 10MB dataset (SWI and Sedna), and the same for the 1GB dataset (YAP and Sedna). First, we used the Shapiro-Wilk normality test to find out if our results followed a normal distribution. The results indicate that they do not ($p\text{-}value < 0.01$). Then, we used the Wilcox paired test to calculate the *p-values*, which are shown in Table 1. Note that the performance difference of SWI and Sedna for the TC/SD 10MB dataset is not statistically significant ($p\text{-}value < 0.05$). However, for all other datasets, it is.

We then proceeded to calculate the Cliff Delta's effect size for the seven scenarios in which we had a significant statistical difference. For the dataset containing all 10MB queries, the effect size is medium ($delta$ = -0.3836986). For the DC/SD 10MB dataset, the effect size is large ($delta$ = -0.609375), and for the TC/MD 10MB dataset, the effect size is medium ($delta$ = -0.4487535). For the 1GB datasets, the results are as follows: large effect size for the TC/SD dataset ($delta = 0.5155709$); medium effect size for the DC/SD dataset ($delta = 0.359375$); negligible effect size for the TC/MD dataset ($delta$ = -0.06371191); and small effect size for the full dataset ($delta = 0.2683583$). This means that although YAP performed better than Sedna in the TC/MD dataset, the difference is not significant.

RQ1: Are Prolog engines comparable to XQuery engines in terms of performance?

Answer: For small datasets, the query processing times of the Prolog queries are competitive, and even better than those obtained with the native XML query processors in some cases (SWI performs better than any other approach in 27 out of 51 queries in the 10MB dataset, which represents 52% of the queries of this dataset). Our analysis shows a significant statistical difference with a medium effect size when we consider all the 51 queries of the 10MB dataset. Also, for the TC/SD 10MB dataset, there was no statistically significant difference in the results. Additionally, for large datasets (1GB), the best option is still to use XML engines. Although YAP performed better than Sedna in the TC/MD dataset, the effect size is negligible.

The answer to this question leaves space for RQ2, in which we investigate when it is a good choice to use Prolog engines to run XML queries.

5.5. RQ2: When is it a good choice to use Prolog engines to process XQuery queries?

To answer RQ2, we analyze the queries of the datasets in which we had significantly statistical difference in the results, and select XQuery features that may affect the query processing time. The basic structure of an XQuery query is a FLWOR, which is an abbreviation of *for, let, where, order by, return*. We analyze each query and annotate which XQuery construct they use to help us answer RQ2. Although we translated the queries to Prolog in an ad-hoc manner, we kept the translation consistent – queries with the same features were translated using the same "algorithm." This is why we can make this analysis by query feature.

Depending on the query processing times, and the features each query exercises, we define a set of heuristics to answer RQ2. Tables 2 and 3 show the features of each query for the DC/SD and TC/MD datasets, which are the ones in which we had significant statistical differences in the results. Tables 2 and 3 have 9 columns, which we detail below.

Table 2: XQuery technical features for queries over the DC/SD dataset – the output size is calculated using the 1GB dataset

	for	for filter	let	let filter	where	max depth path	join	order by	return	output size (bytes)
Q01	x	key				2			1 elt. type	5.4 K
Q02	x				=	6			1 elt. type	0
Q03	distinct()	>=, <	x	=		5	let filter		1 elt. type + aggregate	342 K
Q04	x	position()	x	key		2	for filter by previous sibling		2 elt. types	69
Q05	x	key				2			1 elt. type	1.2 K
Q06	x				some	7			1 elt. type	36 M
Q07	x				every	7			1 elt. type	1.6 M
Q08	wildcard in path	key				2			1 elt. type	765
Q09	x				key	2			1 elt. type, descendant in path	13
Q10	x				conjunction, >, <	3		x	2 elt. types, order by	12 M
Q11	x				conjunction, >, <	3		x	2 elt. type, order by descending	1.9 M
Q12	x	key				2			1 elt.type, position()	399
Q14	x				conjunction, <, >, empty()	5			1 elt. type	106 K
Q17	x				contains()	3			1 elt. type	1.2 M
Q19	x	key			=	5	where clause		1 elt. type	397
Q20	x				multiplication, >	4			1 elt. type, ascending in path	644 K

for. The XQuery *for* clause iterates over a set of elements, as a *for* in a programming language does. It takes a variable and a path, and, at each iteration, links the next element that matches the path into the variable. In the example query of Figure 18, the `$item` variable iterates over the path `catalog/item[@id="I7"]`. In some queries, there may be a *distinct()* function applied over the path on the *for* clause, meaning that only distinct elements matching that path will be iterated over.

let. Differently from the *for* clause which iterates over the elements that match a path, the *let* clause matches all elements of a path to a variable at once. This means that this clause is executed only once, unless it is placed after a *for* clause.

Table 3: XQuery technical features for queries over the TC/MD dataset – the output size is calculated using the 1GB dataset

	for	for filter	let	let filter	where	max depth path	join	order by	return	output size (bytes)
Q01	x	key				1			1 elt. type	62
Q02	x				=	5			1 elt. type	0
Q03	distinct()		x	=		4	let filter		1 elt type + aggregate	181 K
Q04	x	key, =				4	for filter by following sibling		1 elt. type	0
Q05	x	key				1			1 elt type, position()	70
Q06	x				some, conjunction of contains()	4			1 elt. type	35 K
Q07	x				every, contains()	4			1 elt. type	46 K
Q08	x	key				1			1 elt type, wildcard in path	969
Q09	x	key				1			1 elt type, descendant in path	499
Q10	x					4		x	2 elt types	316 K
Q11	x				=	4		x	2 elt types	703
Q12	x	key				1			1 elt. type	158 K
Q13	x	key				1			4 elt. types, position	1,4 K
Q14	x				empty()	2			1 elt. type	112 K
Q15	x				empty()	4			1 elt. type	3.3 M
Q16	x	key				1			1 elt. type	298 K
Q17	x				descendant, contains()	?			1 elt. type	183 K
Q18	x				descendant, contains()	?			2 elt. types	1.2 M
Q19	x	key				4	where clause		1 elt. type	201 K

for and let filters. The *for* and *let* clauses may have one of more filters (expressed in brackets `[]`) in their path expression. Whenever this filter is an equality that compares a key element or attribute with a constant value, we consider this query to search for a key (and this is marked as *key* in Tables 2 and 3). This happens, for instance, in the query of Figure 18, where the first *for* clause has a filter that retrieves items by the key *id = "I7"*.

where. The *where* clause filters the query answer. It is a boolean expression that, when evaluated to True, triggers the execution of the *return* clause of the query. This clause may have functions, such as *contains()* or *empty()*, relational

```
for $item in input()/catalog/item[@id="I7"],
    $related in input()/catalog/item
where $item/related_items/related_item/item_id = $related/@id
return
<Output>
  {$related/title}
</Output>
```

Figure 18: Example of query from the benchmark - Q19 of DC/SD dataset joins items with related items

operators (equal (=), not equal (! =), greater than (>), greater than or equal to (>=), lower than (<) and lower than or equal to (<=)), and quantifiers. The universal quantifier (*every*) is used to retrieve a set of elements when all of them satisfy a given condition, while the existential quantifier (*some*) is used to retrieve a set of elements when at least one of them satisfies a given condition.

max depth path. The *max depth path* column on Tables 2 and 3 register the max depth path of the query. This path may be at the *for*, *let*, *where* or *order by* clause. Queries Q17 and Q18 of the TC/MD dataset have an *?* at this column, since the path uses a descendant axis to find *p* elements in any depth of the document.

join. A query contains a join if it relates two different parts of the document or two different documents. Thus, even queries over SD datasets may have joins, as shown in Table 2. An example of such query is shown in Figure 18, which shows query Q19 of the DC/SD dataset. This query joins items with related items by their *id*.

return. The return clause contains elements that will be returned by the query. In Tables 2 and 3, the *return* column describes the number of element types a given query returns. It also mentions whether the query contains aggregates, the *position()* function, or access to descendant or ancestors of elements.

order by. The *order by* operator sorts the query results by a given element (or sets of elements).

output size. The output size of the queries is calculated using the 1GB dataset.

Figure 10 shows that SWI performs better than Sedna for almost every query in the DC/SD 10MB dataset. The exceptions are queries Q03 and Q20. Q3

is the only query that has a *let* preceded by a *for*. This means that for each element over which the *for* iterates, the query processor needs to calculate the path on the *let* clause. Depending on the size of this list, this takes a long time in Prolog. Additionally, this query has a *distinct* operator, which requires additional processing to remove duplicates in the elements that match the *for* clause. Query Q20 does a multiplication in the *where* clause, and compares the resulting value with a constant. Queries with these features perform better in XML engines. The queries for which Prolog performs much better than XML engines are those that have a filter or *where* clause that searches for a key or by the element position.

To answer a query, Prolog's inference engines rely on SLD-resolution [39] and depth-based search. Considering the search space of such a query built upon a tree, we have the negation of the atoms of the query composing the root of the tree. Next, the inference engine searches for some clause that can *unify* with the first atom in the node, using the resolution rule. Then, new nodes are added to such a tree, where a new node is the result of a unification between the head of the clause and the previous node, and the edge is annotated with such a clause and the unifier. By applying the resolution rule, the head of the clause and the first atom of the query "cancel" each other. The search continues until it reaches an empty node (which can only be a leaf), when we say that a refutation occurred and the query is proved by refutation. In case this empty node is not found, we cannot say that the query is proved. When the search is for a specific term (a constant), the answer encompasses the unification that has been annotated at some edge in the path between the root and the empty leaf. Because of such unifications yielded from applying the resolution rule, the inference engines have the ability to find constants that can reside at any point of a hierarchical structure. This is one of the reasons that makes searching elements by a key value or by its position more efficient in Prolog engines than in XML-based searches.

Consider, for example, the Prolog translation exhibited in the Figure 4 and assume that one would like to know the name of John's child. In this

Table 4: Heuristic Map

Dataset Size	Dataset Type	where or filter clause	Best Approach
Small	DC/SD	search by key or position()	Prolog Engine
		other	XML Engine
	TC/MD	search for substrings using contains()	XML Engine
		other	Prolog Engine
Large	Any	Any	XML Engine

case, the query would be composed by the atoms: `father(ID1, _, 'John')`, `name(ID1, _, Child)`. The query procedure would start by resolving `father(ID1, _, 'John')` with `father(2, 6, 'John')`, and the variable `ID1` would be replaced by `2`, resulting in `name(2,_, Child)`. Next, `name(2, _, Child)` would be resolved with `name(2, 4, 'Mary')`, unifying the variable `Child` with `'Mary'` and resulting in an empty clause. We can observe that by relying on such unifications, Prolog engines can find the answer for key terms directly from its tree search mechanism without traversing all the possible parents and hierarchical structures of the document.

In the TC/MD 10MB dataset, SWI performs better that Sedna in 14 out of 19 queries. The exceptions were Q6, Q7, Q16, Q17, and Q18. All of these queries, except for Q16, searches for substrings using the *contains* function, which is an expensive operation in Prolog, since it requires examining all the facts. Q16 is the only query in this set that searches by key. However, the difference in the query processing time when we compare Sedna and SWI for this query is minimal, thus we do not consider that this query is an exception to the rule that search by key is efficient in Prolog, in general.

RQ2: When is it a good choice to use Prolog-based approaches to process XQuery queries?

Answer: Using as basis a set of XQuery features, we have analyzed the query processing time of 35 queries from the DC/SD and TC/MD datasets. Based on these results, we have detected a key scenario in which it is a good idea to

use Prolog engines to process XML queries: when the dataset is small, and the query searches by key or position. For large datasets, or queries that search for substrings, XML engines are the best option. Table 4 summarizes the results and shows a set of heuristics to determine the better mechanism to process queries based on the characteristics of the queries and datasets.

---

6. Final Remarks

In this work, we investigate the use of Prolog engines to process XML queries. Our goal is to evaluate whether the effort of building an algorithm to automatically translate XQuery queries to Prolog would be worth it. Our evaluation was guided by two research questions that were answered using the results of the experiments.

We conclude that, in some cases, it is worth to use Prolog engines to process queries in XQuery. Thus, the effort to build an automatic translation algorithm may be worth it. However, there are several aspects that need to be noted. First, our experiments do not take into account the time to translate the source XML documents into Prolog facts. In fact, using Prolog to process XML queries would only pay off in case of read-only datasets, or datasets that change very infrequently. Second, we also do not take into consideration the time needed to translate the XQuery queries into Prolog rules. This is because our queries were translated manually, and so it would not be possible to accurately measure this time. Any algorithm to automatically translate XQuery queries into Prolog would need to be evaluated to consider this translation time. Third, the benchmark we used is not representative of all possible applications. There may be several applications with queries with different features that we did not evaluate due to this reason. As an example, all the queries of the benchmark are fast. Results may differ for long-running queries. Fourth, our results for the 1GB datasets were not good. This is a clear indication that our approach is not meant for large datasets. However, according to [34], XML documents tend to

be not so large, and thus we believe the results we got for the 10MB datasets are meaningful. Finally, we have used three very well known native XML databases in our evaluation, but there may be others for which query processing times are lower. In spite of that, we do believe that the results we present here are a good first step into the subject.

The findings of our work have several theoretical and practical implications. In terms of theory, it opens space for automatic XQuery to Prolog translation algorithms. In fact, the translation of XQuery queries into Prolog rules is a very time-consuming step. For now, this task is manual and hard. Also, our results show the need for heuristic algorithms that help decide when it is a good idea to use the translation to Prolog. The results we describe in this paper can be used as the first basis for those heuristic algorithms. As for the practical implications, a solution that uses Prolog to execute XML queries opens the opportunity to query not only explicit information, but also implicit ones, obtained by inference. In fact, our experiments with a recursive query show that Prolog engines are two orders of magnitude faster than the XML engines we tested.

Acknowledgments

The authors would like to thank CNPq for partially supporting this research work.

References

[1] Abiteboul, S., Hull, R., Vianu, V., 1996. Foundations of Databases. Addison-Wesley.

[2] Afanasiev, L., Franceschet, M., Marx, M., 2006. XCheck: A Platform for Benchmarking XQuery Engines. In: International Conference on Very Large Data Bases (VLDB). VLDB Endowment, pp. 1247–1250, event-place: Seoul, Korea.

[3] Agarwal, M. K., Ramamritham, K., Agarwal, P., 2016. Generic Keyword Search over XML Data. In: International Conference on Extending Database Technology (EDBT). pp. 149–160.

[4] Alghamdi, N. S., Rahayu, W., Pardede, E., 2014. Semantic-based Structural and Content Indexing for the Efficient Retrieval of Queries over Large XML Data Repositories. Future Generation Computer Systems (FGCS) 37, 212–231.

[5] Almendros-Jiménez, J. M., Becerra-Terón, A., Enciso-Baños, F. J., 2008. Querying XML documents in logic programming. Journal of Theory and Practice of Logic Programming 8 (3), 323–361.

[6] Almendros-Jiménez, J. M., Becerra-Terón, A., Enciso-Baños, F. J., 2009. Integrating XQuery and Logic Programming. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 117–135.

[7] Almendros-Jimenez, J. M., Caballero, R., Garcia-Ruiz, Y., Saenz-Perez, F., 2011. Xquery in the functional-logic language toy. In: Kuchen, H. (Ed.), Functional and Constraint Logic Programming. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 35–51.

[8] Almendros-Jimenez, J. M., Caballero, R., Garcia-Ruiz, Y., Saenz-Perez, F., 2012. Xpath query processing in a functional-logic language. Electronic Notes in Theoretical Computer Science 282, 19–34.

[9] Almendros-Jiménez, J. M., Tedesqui, A. L., Moreno, G., 2015. Fuzzy xpath through fuzzy logic programming. New Generation Computing 33 (2), 173–209.

[10] Alrammal, M., Hains, G., Jan. 2014. A Research Survey on Large XML Data: Streaming, Selectivity Estimation and Parallelism. In: Xhafa, F., Bessis, N. (Eds.), Inter-cooperative Collective Intelligence: Techniques and Applications. No. 495 in Studies in Computational Intelligence. Springer Berlin Heidelberg, pp. 167–202.

[11] Alves, M. B., Damásio, C. V., Correia, N., 2016. Rdf query and inference in prolog. In: International Conference on Knowledge Engineering and the Semantic Web. Springer, pp. 191–201.

[12] Bailey, J., Bry, F., Furche, T., Schaffert, S., 2005. Web and semantic web query languages: a survey. In: Eisinger, N., Maluszynski, J. (Eds.), Reasoning Web. Vol. 3564 of Lecture Notes in Computer Science. Springer, pp. 35–133.

[13] Bamford, R., Borkar, V., Brantner, M., Fischer, P. M., Florescu, D., Graf, D., Kossmann, D., Kraska, T., Muresan, D., Nasoi, S., Zacharioudakis, M., Aug. 2009. XQuery reloaded. Proceedings of the VLDB Endowment (PVLDB) 2 (2), 1342–1353.

[14] Bača, R., Krátký, M., Holubová, I., Nečaský, M., Skopal, T., Svoboda, M., Sakr, S., 2017. Structural XML Query Processing. ACM Computing Surveys (CSUR) 50 (5), 64:1–64:41.

[15] Boag, S., Chamberlin, D., Fernandez, M., Florescu, D., Robie, J., Simeon, J., 2010. XQuery 1.0: an XML query language (Second Edition). W3C Recommendation. URL http://www.w3.org/TR/xquery/.

[16] Boley, H., 2000. Relationships between logic programming and XML. In: Workshop on Logic Programming. Berlin, Germany, pp. 19–34.

[17] Boncz, P., Grust, T., van Keulen, M., Manegold, S., Rittinger, J., Teubner, J., 2005. Pathfinder: XQuery – the Relational Way. In: International Conference on Very Large Data Bases (VLDB). VLDB Endowment, pp. 1322–1325, event-place: Trondheim, Norway.

[18] Braganholo, V., Mattoso, M., Dec. 2014. A Survey on XML Fragmentation. SIGMOD Record 43 (3), 24–35.

[19] Bry, F., Schaffert, S., 2003. The xml query language xcerpt: Design principles, examples, and semantics. In: Chaudhri, A. B., Jeckle, M., Rahm, E.,

Unland, R. (Eds.), Web, Web-Services, and Database Systems. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 295–310.

[20] Buneman, P., Fernandez, M. F., Suciu, D., 2000. UnQL: A query language and algebra for semistructured data based on structural recursion. VLDB Journal 9 (1), 76–110.

[21] Clark, J., Derose, S., 1999. XML Path Language (XPath) Version 1.0. W3C Recommendation. URL http://www.w3.org/TR/xpath/.

[22] Coelho, J., Florido, M., 2003. Type-based XML processing in logic programming. In: International Symposium on Practical Aspects of Declarative Languages. New Orleans, USA, pp. 273–285.

[23] Coelho, J., Florido, M., 2007. Xcentric: logic programming for XML processing. In: International Workshop on Web Information and Data Management. Lisbon, Portugal, pp. 1–8.

[24] Costa, V. S., Rocha, R., Damas, L., Jan. 2012. The Yap Prolog System. Theory Practice of Logic Programming 12 (1-2), 5–34.

[25] Dahaka, F., Boughanemb, M., Ballaa, A., 2017. A probabilistic model to exploit user expectations in XML information retrieval. Information Processing and Management 53, 87–105.

[26] de Campos, L. M., Fernández-Luna, J. M., Huete, J. F., Martín-Dancausa, C. J., 2010. Managing structured queries in probabilistic XML retrieval systems. Information Processing and Management 46 (5), 514–532.

[27] Fana, H., Maa, Z., Wang, D., Liu, J., 2018. Handling distributed xml queries over large xml data based on mapreduce framework. Information Sciences 453, 1–20.

[28] Fegaras, L., 2016. Incremental query processing on Big Data streams. IEEE Transactions on Knowledge and Data Engineering 28 (11), 2998–3012.

[29] Fernandez, M., Simeon, J., Choi, B., Marian, A., Sur, G., 2003. Implementing XQuery 1.0: the Galax Experience. In: International Conference on Very Large Data Bases. Vol. 29. Berlin, Germany, pp. 1077–1080.

[30] Fernandez, M., Simeon, J., Wadler, P., 2000. An algebra for XML query. In: International Conference on Foundations of Software Technology and Theoretical Computer Science. pp. 11–45.

[31] Florescu, D., Kossmann, D., Manolescu, I., 2000. Integrating keyword search into xml query processing. Computer Networks 33 (1-6), 119–135.

[32] Fomichev, A., Grinev, M., Kuznetsov, S., 2006. Sedna: A Native XML DBMS. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 272–281.

[33] Frasincar, F., Houben, G.-J., Pau, C., 2002. XAL: an algebra for XML query optimization. Australasian Computer Science Communications 24 (2), 49–56.

[34] Grijzenhout, S., Marx, M., 2011. The Quality of the XML Web. In: International Conference on Information and Knowledge Management (CIKM). ACM, New York, NY, USA, pp. 1719–1724, event-place: Glasgow, Scotland, UK.

[35] Grün, C., Holupirek, A., Scholl, M., 01 2007. Visually exploring and querying xml with basex. Datenbanksysteme in Business, Technologie und Web (BTW) 12, 629–632.

[36] Jagadish, H. V., Lakshmanan, L. V. S., Srivastava, D., Thompson, K., 2001. TAX: a tree algebra for XML. In: International Workshop on Database Programming Languages. pp. 149–164.

[37] Kotsakis, E., 2002. Structured Information Retrieval in XML Documents. In: ACM Symposium on Applied Computing. Madrid, Spain, pp. 663–667.

[38] Lima, D., Delgado, C., Murta, L., Braganholo, V., 2012. Towards Querying Implicit Knowledge in XML Documents. Journal of Information and Data Management 3 (1), 51–60.

[39] Lloyd, J. W., 1984. Foundations of Logic Programming. Springer-Verlag, Berlin, Heidelberg.

[40] Lopes, C., Knorr, M., Leite, J., 2017. Nohr: integrating xsb prolog with the owl 2 profiles and beyond. In: International Conference on Logic Programming and Nonmonotonic Reasoning. Springer, pp. 236–249.

[41] Manegold, S., 2008. An empirical evaluation of XQuery processors. Information Systems 33 (2), 203–220.

[42] Meier, W., 2003. eXist: An Open Source Native XML Database. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 169–183.

[43] Minker, J., 1999. Logic and Databases: a 20 Year Retrospective - Updated in Honor of Ray Reiter. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 234–299.

[44] Morales, J. F., Hermenegildo, M., 2015. Pre-indexed terms for prolog. In: Proietti, M., Seki, H. (Eds.), Logic-Based Program Synthesis and Transformation. Springer International Publishing, Cham, pp. 317–331.

[45] Niemi, T., Jarvelin, K., 1991. Prolog-based meta-rules for relational database representation and manipulation. IEEE Transactions on Software Engineering 17 (8), 762–788.

[46] Paparizos, S., Wu, Y., Lakshmanan, L. V. S., Jagadish, H. V., 2004. Tree logical classes for efficient evaluation of XQuery. In: SIGMO. pp. 71–82.

[47] Robie, J., Derksen, E., Fankhauser, P., Howland, E., Huck, G., Macherius, I., Murata, M., Resnick, M., Schöning, H., 1999. XQL: XML Query Language. URL `http://www.ibiblio.org/xql/xql-proposal.html`.

[48] Robie, J., Dyck, M., Spiegel, J., 2017. XML Path Language (XPath) 3.1. W3C Recommendation. URL `https://www.w3.org/TR/xpath-3/`.

[49] Robie, J., Dyck, M., Spiegel, J., 2017. XQuery 3.1: An XML Query Language. W3C Recommendation. URL https://www.w3.org/TR/xquery-31/.

[50] Santos, F., Jul. 2015. Prolog versus XQuery Processors: a performance evaluation of XML Queries Processing Methods. Master thesis, Universidade Federal Fluminense, Niterói, RJ, Brasil, URL http://www.ic.uff.br/PosGraduacao/frontend-tesesdissertacoes/download.php?id=693.pdf&tipo=trabalho.

[51] Santos, F., Pinheiro, R., Braganholo, V., 2012. Processamento de Consultas XML usando Máquinas de Inferência (in Portuguese). In: Brazilian Symposium on Databases (SBBD). São Paulo, Brasil, pp. 129–136.

[52] Sartiani, C., Albano, A., 2002. Yet another query algebra for XML data. In: Database Engineering and Applications Symposium. pp. 106–115.

[53] Seipel, D., 2002. Processing XML-documents in Prolog. In: Workshop Logische Programmierung. Dresden, Germany, pp. 1–15.

[54] Strate, J., Fritchey, G., 2015. XML Indexes. Apress, Berkeley, CA, pp. 101–114.

[55] Suryanarayana, D., Hussain, S. M., Kanakam, P., Gupta, S., 2018. Natural language query to formal syntax for querying semantic web documents. In: Saeed, K., Chaki, N., Pati, B., Bakshi, S., Mohapatra, D. P. (Eds.), Progress in Advanced Computing and Intelligent Engineering. Springer Singapore, Singapore, pp. 631–637.

[56] Tahraoui, M. A., Pinel-Sauvagnat, K., Laitang, C., Boughanem, M., Kheddouci, H., Ning, L., 2013. A survey on tree matching and xml retrieval. Computer Science Review 8, 1–23.

[57] team at the Centrum Wiskunde & Informatica (CWI), M., 2006. Comparison of MonetDB with other XQuery systems and previous

MonetDB versions. MonetDB WebsiteURL https://www.monetdb.org/XQuery/Benchmark/XMark/.

[58] Trotman, A., May 2004. Searching structured documents. Information Processing and Management 40 (4), 619–632.

[59] Wielemaker, J., Jan. 2002. A C++ Interface to SWI Prolog. URL http://www.swi-prolog.org/pldoc/doc_for?object=section(%27packages/pl2cpp.html%27).

[60] Wielemaker, J., Nov. 2009. SWI-Prolog SGML/XML parser. URL http://www.swi-prolog.org/pldoc/doc_for?object=section(%27packages/sgml.html%27).

[61] Wielemaker, J., Schrijvers, T., Triska, M., Lager, T., Jan. 2012. Swi-prolog. Theory Pract. Log. Program. 12 (1-2), 67–96.

[62] Yao, B., Ozsu, T., Khandelwal, N., 2004. XBench Benchmark and Performance Testing of XML DBMSs. In: International Conference on Data Engineering. Boston, MA, USA, pp. 621–632.

[63] Yun, J., Chung, C., 2012. Efficient probabilistic XML query processing using an extended labeling scheme and a lightweight index. Information Processing and Management 48 (6), 1181–1202.

[64] Zhang, M., Yao, J. T., 2004. XML algebras for data mining. In: Data Mining and Knowledge Discovery: theory, tools and technology. pp. 209–217.

[65] Zhang, X., Pielech, B., Rundesnteiner, E. A., 2002. Honey, i shrunk the XQuery!: an XML algebra optimization approach. In: WIDM. pp. 15–22.

[66] Zhou, J., Wang, W., Chen, Z., Yu, J. X., Tang, X., Lu, Y., Li, Y., 2016. Top-down XML Keyword Query Processing. IEEE Transactions on Knowledge and Data Engineering (TKDE) 28 (5), 1340–1353.

[67] Ziyang Liu and Yi Chen, 2011. Processing Keyword Search on XML: a survey. In: World Wide Web. pp. 671–707.